A Layered Architecture for Governed MCP

The SecureMCP Design

Fourteen opt-in layers, five enforcement middleware, and one bootstrap that wires them together.

---

1. Thesis

The Model Context Protocol solved *exposure*. A server declares tools, resources, and prompts; a client discovers and invokes them. FastMCP made that exposure ergonomic — decorate a Python function, get an MCP tool.

What the protocol does not define is *governance*. MCP has no notion of who may call a tool, under what terms, with whose consent, subject to which regulation, with what tamper-evident record, or what should happen when an agent's behaviour changes shape mid-session. Those questions are left entirely to the server author, which in practice means they are answered ad hoc or not at all.

SecureMCP is the layer that answers them. The one-line framing:

> **FastMCP exposes capability. SecureMCP governs capability.**

SecureMCP is not a fork of FastMCP and not a proxy in front of it. It is a subclass, a middleware chain, and fourteen independently-enableable subsystems, wired together by a single stateless bootstrap function. What follows is the architecture as built: the request path, the data structures, the failure modes, and the points where the design deliberately stops short of what a reader might otherwise assume.

Scope. The subject is the SecureMCP server framework — `fastmcp_slim/fastmcp/server/security/` and the `securemcp` facade at `src/securemcp/`. The separate product layers that ship from the same source tree are out of scope.

---

2. Design Principles

Five commitments govern nearly every implementation decision in the platform.

Additive, not invasive. SecureMCP must never require a change to FastMCP core semantics. Where SecureMCP needs per-server state, it lives in module-level `WeakKeyDictionary` maps keyed by the server object — never in fields FastMCP owns. A FastMCP server with security attached is still a FastMCP server, and the security state is garbage-collected with it.

Opt-in per layer. There is no monolithic "security mode." Each of the fourteen subsystems is enabled if and only if its config object is non-`None` *and* the master switch is on. The predicate is uniform:

```
def is_policy_enabled(self) -> bool:
    """Check if the policy kernel is configured and active."""
    return self.enabled and self.policy is not None
```

A server can run the provenance ledger with nothing else, or the policy kernel with nothing else, and pay only for what it turns on.

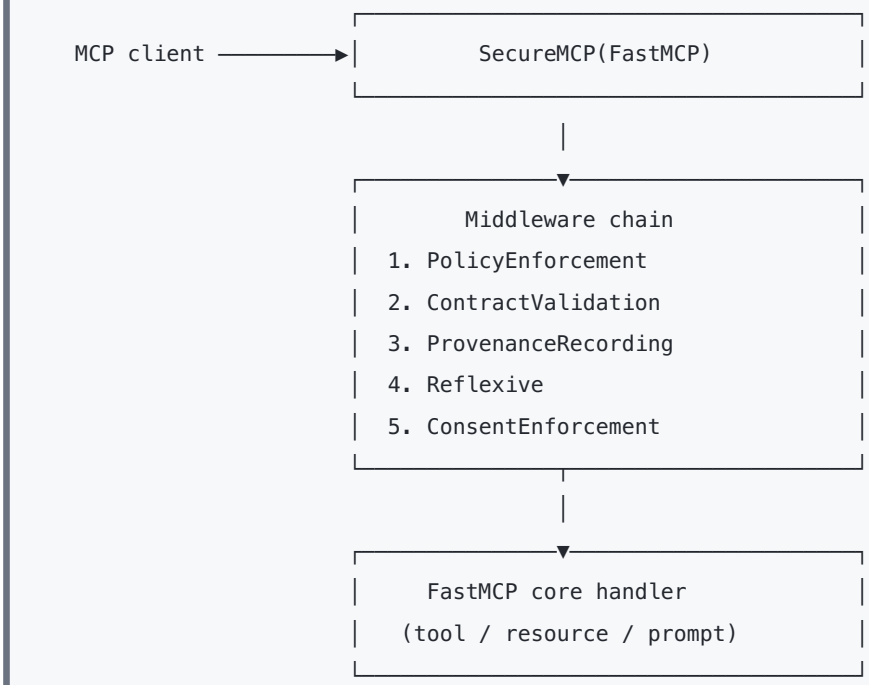
Fail closed on the enforcement path. When the policy kernel cannot reach a decision — a provider raised, the request context is missing, every provider deferred — the default outcome is DENY. This is a configurable default (`fail_closed=True`), not a hard rule, but the enforcement path is written so that the safe outcome is the one that requires no correct behaviour from anything else.

Explainability as a first-class output. Every decision carries structured provenance for *why*. A policy result carries a reason, a `policy_id`, and a deduplicated list of `Citation` objects pointing at the regulation clause that produced it. A consent decision carries the full graph path that granted or denied it. A drift event carries the metric, the baseline, and the sigma distance. The system is designed to be auditable by someone who was not present when the decision was made.

Honest about its boundaries. The platform is deliberately explicit about what it does *not* enforce. The sandbox module's own docstring states that it provides no kernel boundary. Section 15 treats this in full, because a governance layer that overstates its guarantees is worse than one that claims fewer.

---

3. Architecture at a Glance




```
Bootstrapped by SecurityOrchestrator into a SecurityContext:

Event Bus → Policy Kernel → Context Broker → Provenance →
Reflexive → Consent → Trust Registry → CRL → Federation →
Tool Marketplace → Compliance → Sandbox → Certification →
Gateway → Dashboard
```

3.1 The orchestrator / context pattern

Wiring security once meant a long stretch of inline conditional construction inside `server.py`. That has been replaced by a single stateless factory:

```
ctx = SecurityOrchestrator.bootstrap(security_config, server_name="my-server")
middleware = ctx.middleware
```

`SecurityOrchestrator.bootstrap()` is a classmethod with no instance state. It reads a `SecurityConfig`, walks the fifteen construction blocks in dependency order, and returns a `SecurityContext` — a dataclass whose every field is optional. A field is populated if and only if its layer was enabled.

Three properties follow from this:

- **Testability.** Bootstrap is a pure function from config to context. Tests construct exactly the layers they exercise.
- **Introspectability.** `ctx.policy_engine is None` is the authoritative answer to "is policy on?" There is no second source of truth.
- **Order safety.** Dependencies are expressed by construction order in one readable function rather than scattered across a class hierarchy.

Two details of the bootstrap carry more weight than their size suggests.

Event-bus propagation is *gated*. Components receive the bus only if propagation is enabled:

```
bus_for_components = event_bus if propagate else None
```

Every subsequent `attach_event_bus` call is guarded by `if bus_for_components is not None`. Disabling propagation genuinely detaches the subsystems rather than leaving them emitting into a discarded sink.

The dashboard is always constructed. It is a read-only projection over whatever else exists, so there is no configuration in which asking "what is the security posture of this server?" fails for lack of a dashboard.

3.2 The fourteen layers

Layer	Predicate	Core object	On the request path?
Policy kernel	`is_policy_enabled`	`PolicyEngine`	Yes — middleware 1
Contracts	`is_contracts_enabled`	`ContextBroker`	Yes — middleware 2
Provenance	`is_provenance_enabled`	`ProvenanceLedger`	Yes — middleware 3
Reflexive	`is_reflexive_enabled`	`BehavioralAnalyzer` + `EscalationEngine`	Yes — middleware 4
Consent	`is_consent_enabled`	`ConsentGraph`	Yes — middleware 5
Alerts	`is_alerts_enabled`	`SecurityEventBus`	Indirect
Certification	`is_certification_enabled`	`CertificationPipeline`	No — publish-time
Trust registry	`is_registry_enabled`	`TrustRegistry`	No — query-time
Tool marketplace	`is_tool_marketplace_enabled`	`ToolMarketplace`	No
Federation	`is_federation_enabled`	`TrustFederation`	No
CRL	`is_crl_enabled`	`CertificateRevocationList`	Sandbox start / trust query
Compliance	`is_compliance_enabled`	`ComplianceReporter`	No — reporting
Sandbox	`is_sandbox_enabled`	`SandboxedRunner`	Cooperative only (§15)
Gateway	`is_gateway_enabled`	`AuditAPI`, `Marketplace`	No — observability

4. Integration: Attaching Without Forking

`src/securemcp/server.py` is deliberately thin:

```
class SecureMCP(FastMCP[LifespanResultT], Generic[LifespanResultT]):
    ...
```

It adds a small set of keyword arguments (`security`, `mount_security_api`, `security_api_prefix="/security"`, `security_api_require_auth=True`, `bypass_stdio`, `register_gateway_tools`) and two properties (`security_context`, `security_api`). It raises `ValueError` if `register_gateway_tools=True` is passed without a `security=` config, because gateway tools with nothing to report on is a configuration error rather than a degraded mode.

Everything else lives in `integration.py`, which is where the "additive" principle is mechanised:

```
_ATTACHED_SECURITY_CONTEXTS: WeakKeyDictionary[FastMCP, SecurityContext] =
WeakKeyDictionary()
_REGISTERED_GATEWAY_TOOLS: WeakKeyDictionary[FastMCP, set[str]] = WeakKeyDictionary()
```

This is what makes `attach_security(server, config)` work on a *plain* `FastMCP` instance, not just a `SecureMCP` subclass. Security is a capability attached to a server, and `SecureMCP` is a convenience constructor that attaches it on the caller's behalf.

4.1 Settings precedence

Three sources can configure security, resolved in this order:

1. Explicit keyword arguments to the constructor or `attach_security`
2. Explicitly-set `SECUREMCP_*` environment variables
3. The code-level `SecurityConfig`

"Explicitly set" is load-bearing. `_apply_settings_overrides` consults `settings.model_fields_set` rather than reading values, so a Pydantic default is never mistaken for an operator decision. `SECUREMCP_POLICY_FAIL_CLOSED` and `SECUREMCP_POLICY_HOT_SWAP` (with `FASTMCP_SECURITY_*` aliases retained for compatibility) are honoured, and — importantly — replicated onto a live pre-built engine, not only onto the config that would have built one. If an override has nothing to bind to, the code warns rather than failing silently.

4.2 The STDIO warning

`bypass_stdio` exists because STDIO transport historically implies a trusted local parent process. But bypassing policy on STDIO while believing policy is enforced is exactly the kind of gap that governance layers are supposed to close, so attaching with `bypass_stdio=True` and policy enabled emits a loud `WARNING` once per attach.

The default across all five middleware is `bypass_stdio=False`. STDIO is treated as a privileged execution surface that still deserves an audit trail.

---

5. The Request Path

This is where the architecture earns its keep, because it is what runs on every call.

The orchestrator appends middleware in a fixed order, and that order is the enforcement semantics:

```
PolicyEnforcement → ContractValidation → ProvenanceRecording → Reflexive → ConsentEnforcement
→ core handler
```

The sequence is deliberate:

1. **Policy first.** The cheapest and most categorical rejection happens before anything else does work.
2. **Contracts second.** Terms of engagement are checked once the caller is known to be permitted in principle.
3. **Provenance third.** Positioned so that it records operations that *passed* admission control, and — because it wraps `call_next` in `try/except` — records failures from everything downstream of it, including the handler itself.
4. **Reflexive fourth.** Behavioural observation and pre-execution gating, which needs to be close to actual execution to measure it.
5. **Consent last.** The final owner-authorisation check immediately before the handler runs.

5.1 Actor identity

All five middleware derive actor identity the same way, and all five redact it:

```
token = get_access_token()
if token is not None:
    return token.token[:8] + "..."
return "anonymous"
```

An eight-character prefix is enough to correlate a session across the ledger, the drift baselines, and the consent audit log, without writing bearer tokens into a durable append-only store. It is a correlation identifier, not a credential.

5.2 Policy enforcement middleware

The first middleware is fail-closed at the outermost level: if `fastmcp_ctx is None`, it denies. A request it cannot describe is a request it will not authorise.

Its most important behaviour is that it **rebuilds** the evaluation context rather than trusting what arrived. It re-fetches the tool from the server and derives the capability fields from the tool's own tags, using a prefix convention:

```
_READ_ONLY_TAGS: frozenset[str] = frozenset({"read_only", "readonly", "safe"})
_TAG_PREFIX_RESOURCE_TYPE = "resource:"
_TAG_PREFIX_ENVIRONMENT   = "env:"
_TAG_PREFIX_RISK          = "risk:"
_TAG_PREFIX_PRINCIPAL_TYPE = "principal:"
```

So a tool tagged `resource:phi`, `env:production`, `risk:high` is evaluated as a high-risk PHI operation in production regardless of what the caller asserted.

The asymmetry here is deliberate and documented in the source: **tags can describe a tool, but tags cannot claim approval.** The approval fields on the evaluation context are only ever populated from request metadata. A tool author cannot self-authorise by tagging themselves as approved.

Both `DENY` and `REQUIRE_APPROVAL` raise `PolicyViolationError` for execution operations. `REQUIRE_APPROVAL` is a denial *with a remedy* — the error tells the caller what approval to obtain — not a soft warning.

List operations behave differently. They call `call_next` first, then filter the result, removing any component that evaluates to DENY or REQUIRE_APPROVAL, and removing any component whose evaluation raised. Callers do not see the existence of tools they cannot use. Discovery therefore reflects authorisation rather than leaking the catalogue.

Post-decision, the middleware enforces the constraints the policy attached:

Constraint	Effect
`read_only`	Rejects the call unless the tool is tagged read-only
`max_args:N`	Rejects calls with more than N arguments
`require_metadata:KEY`	Rejects calls whose metadata lacks KEY
`log_access`	Emits an access log line

Unknown constraints are debug-logged and non-blocking. This is a considered trade-off: a policy provider that emits a constraint this version does not understand will not brick the server, but it also will not be enforced. Operators should treat unrecognised-constraint debug lines as a version-skew signal.

5.3 Contract validation middleware

Resolves the agent's active contract via `broker.get_active_contracts_for_agent(agent_id)` and checks the requested action against each term's `constraint` dictionary:

- `denied_actions` — action present ⇒ denied
- `allowed_actions` — present and action absent ⇒ denied (allow-list semantics)
- `denied_resources` / `allowed_resources` — same shape for resource identifiers

Absent a valid contract, the middleware fails closed. `require_for_list` defaults to `False`, so listing is unrestricted by contracts unless explicitly configured otherwise.

5.4 Provenance recording middleware

Wraps `call_next` and records on both paths. Success:

```
self.ledger.record(
    action=ProvenanceAction.TOOL_CALLED,
    actor_id=actor_id,
    resource_id=tool_name,
    input_data={"tool": tool_name, "arguments": arguments},
    output_data={"status": "success"},
    metadata={"input_tokens": input_tokens, "output_tokens": output_tokens},
)
```

Failure records `ProvenanceAction.ERROR` with `error` and `error_type`, then **re-raises**. Recording never swallows an exception, and a failed operation is as much a matter of record as a successful one.

Note what is recorded and what is not: inputs are captured in full, outputs are reduced to `{"status": "success"}`. The ledger is a record that an operation occurred with given inputs, not a copy of the data it returned — which keeps result payloads out of a durable append-only structure.

Token counts come from a `chars // 4` heuristic (`_estimate_call_tokens`). Useful for cost attribution and volumetric drift detection; not a substitute for a tokeniser.

`record_list_operations` defaults to `False` — discovery traffic would otherwise dominate the ledger.

5.5 Reflexive middleware

Two modes, depending on whether an `IntrospectionEngine` was configured.

Monitoring-only (no engine): feeds operation metrics — call frequency from `_call_timestamps`, error rates, latency — to the `BehavioralAnalyzer`, and routes resulting drift events to the `EscalationEngine`.

Gating (engine attached): performs a **pre-execution** check and acts on the verdict:

Verdict	Middleware behaviour
`PROCEED`	Continue normally
`REQUIRE_CONFIRMATION`	Raise `ConfirmationRequiredError`
`THROTTLE`	Sleep `throttle_delay_seconds` (default 2.0), then continue
`HALT`	Block the operation

The middleware also maintains a `_suspended_actors` set. Once a `SUSPEND_AGENT` or `SHUTDOWN` escalation fires, subsequent requests from that actor are blocked without re-deriving the verdict.

5.6 Consent enforcement middleware

Constructs a `ConsentQuery` from `(resource_owner, actor_id, scope)` and evaluates it against the graph. Scope is derived from operation type: `EXECUTE` for tool calls, `READ` for resource reads and prompt renders, `LIST` for listings (only when `require_for_list=True`).

A denial raises `ConsentRequiredError` carrying `source_id`, `target_id`, `scope`, and the graph's own reason string — so the caller learns which grant is missing, not merely that something was refused.

6. The Policy Kernel

The largest subsystem in the platform, and its primary decision authority.

6.1 Providers and aggregation

Policy is pluggable. A `PolicyProvider` returns a `PolicyResult` carrying one of four decisions:

```
ALLOW · DENY · DEFER · REQUIRE_APPROVAL
```

`PolicyEngine.evaluate()` runs providers and aggregates with AND semantics and this precedence:

1. **`DENY` short-circuits.** Any deny is final; no later provider can overturn it.
2. **`REQUIRE_APPROVAL` wins** in the absence of a DENY — unless approval has already been consumed.
3. **`ALLOW`** if at least one provider allows and nothing denies or requires approval.
4. **`DEFER`-only** ⇒ `fail_closed` decides (default: DENY).

Sentinel `policy_id` values make the non-obvious outcomes diagnosable rather than mysterious: `engine-no-providers` when nothing is registered, `engine-all-deferred` when every provider abstained.

> **Engineering note.** The "approval consumed" case is detected by substring-matching a provider's free-text reason: `"approval" in r.reason.lower()` on an ALLOW result. It works, but it couples aggregation semantics to human-readable prose. A structured flag on `PolicyResult` is the more durable design and is the intended direction.

6.2 Fail-closed on provider error

A provider raising is treated as a denial, not skipped:

```
PolicyResult(
    decision=PolicyDecision.DENY,
    reason="Policy evaluation failed (fail-closed)",
    policy_id="engine-error",
)
```

A broken policy provider therefore reduces availability rather than silently reducing enforcement.

6.3 The evaluation context

`PolicyEvaluationContext` is a frozen dataclass. Its capability fields (`principal_type`, `resource_type`, `environment`, `risk`, `approval_granted`) all have defaults, so capability-aware providers no-op cleanly against call sites that predate them. The defaults are chosen to be safe rather than permissive — `environment` defaults to `ENVIRONMENT_PRODUCTION` so that a forgotten plumbing step errs toward deny-by-default, and `approval_granted` defaults to `False`.

Risk and environment markers are plain strings rather than enums, deliberately: external policy engines (Rego, Cedar) can consume them without importing Python types.

6.4 Citations

```
@dataclass(frozen=True)
class Citation:
    source: str
    article: str
    url: str
    version: str
```

Every decision can carry citations, and the engine deduplicates them by `(source, article, version)`. Two conventions are enforced by documentation: URLs must point at the primary regulation body rather than an aggregator, and `version` records which revision of the text was enforced — so an audit years later can reconstruct not just *that* a rule applied but *which wording* of it did.

Built-in providers: `TagBasedPolicy`, `ActionBasedPolicy`, `GDPRPolicy`, `HIPAAPolicy`.

6.5 Hot swap, versioning, and invariants

Policy sets can be replaced at runtime. `hot_swap` holds an `asyncio.Lock`, emits `POLICY_SWAPPED`, snapshots a new version, and fires invariant verification as a *tracked* fire-and-forget task — tracked so the task is not garbage-collected mid-flight, fire-and-forget so verification does not sit in the swap's critical section.

Around this core sit the governance modules: a declarative policy language, a validator, a simulation harness for testing scenarios against a policy set before promoting it, an invariant checker, an audit log (default cap 10,000 entries), a monitoring surface, and a workbench with bundles, packs, environment profiles, and promotion flows. Versioning, monitoring, and governance are **off by default** — the kernel's default posture is "enforce, with an audit log," and the policy-lifecycle machinery is opt-in.

6.6 Event emission

The engine emits with `await self._event_bus.aemit(...)` rather than the synchronous `emit`. This is intentional: a slow alert subscriber must not be able to stall a `POLICY_DENIED` notification and, through it, the request that produced it.

---

7. Contracts: Negotiated Terms of Engagement

The `ContextBroker` implements a propose / counter / accept protocol under an `asyncio.Lock`.

An agent proposes terms. The broker prepends the server's `default_terms` — so server-side non-negotiables are always present in the term set, not merely hoped for — evaluates the combination, and either accepts, counters, or rejects. Guard rails: `max_rounds=5`, `session_timeout=30 min`, `contract_duration=1 h`.

7.1 Mutual signing

If *both* a `crypto_handler` and an `agent_registry` are available, contracts require countersignature. The state machine is:

```
NEGOTIATING → PENDING_COUNTERSIGN → ACTIVE
```

The server signs first, moving the contract to `PENDING_COUNTERSIGN`. `agent_sign_contract` then verifies the agent's signature via `crypto_handler.verify_with_external_key(...)` against key material from the `AgentKeyRegistry` before flipping the contract to `ACTIVE`. A contract only becomes enforceable once both parties have cryptographically committed to the same terms.

7.2 Non-repudiation

Every step is written to an `ExchangeLog`: `SESSION_STARTED`, `PROPOSAL_RECEIVED`, `COUNTER_SENT`, `COUNTER_RECEIVED`, `ACCEPTED`, `REJECTED`, `CONTRACT_SIGNED`, `AGENT_SIGNED`, `VERIFICATION_FAILED`, `CONTRACT_REVOKED`. The negotiation itself is auditable, not just its outcome.

> **Known divergence.** `_evaluate_terms` swallows evaluator exceptions and returns `(terms, [])` — that is, **it accepts all terms when evaluation fails.** This is fail-*open*, the opposite of the policy engine's default. The practical mitigation is that contracts are checked *after* policy in the middleware chain, so a permissive contract still cannot admit an operation policy denied. The asymmetry is nonetheless real, and it is disclosed here because operators deploying contracts as a primary control need to account for it.

---

8. Provenance: Dual Integrity

The ledger provides two independent integrity structures over the same records.

A SHA-256 hash chain gives sequential tamper-evidence: altering record n invalidates every hash from n onward.

An incremental Merkle tree gives efficient inclusion proofs: a single record can be proven to belong to the ledger without disclosing the rest of it.

8.1 Genesis hardening

```
_LEGACY_GENESIS_SENTINEL = "genesis"
self._genesis_hash = f"genesis-{self.ledger_id}-{secrets.token_hex(32)}"
```

Each ledger's chain is rooted at a random 32-byte nonce. The reasoning is recorded in the source: an attacker who knows the format cannot forge a fresh, internally-consistent chain rooted at a guessable sentinel and pass it off as the real ledger. `verify_chain` still accepts the legacy `"genesis"` sentinel so existing ledgers remain verifiable.

8.2 Merkle mechanics

`_hash_pair` is `sha256((left + right).encode())`. Odd node counts duplicate the last node. The tree tracks a `_dirty` flag and rebuilds lazily, so a burst of appends costs one rebuild rather than one per record. `MerkleProof.verify()` walks sibling hashes using explicit `"left"` / `"right"` direction markers.

8.3 Exportable verification bundles

`export_verification_bundle(record_id)` is the piece that makes the ledger externally auditable. It emits:

- the record itself
- its Merkle proof (leaf, sibling hashes, directions, root)
- its chain predecessor and successor hashes
- the ledger root and record count

An auditor can recompute both the inclusion proof and the local chain linkage **without access to the running server**. Integrity is verifiable by a third party, not asserted by the system that produced the record.

Pluggable schemes support both local Merkle operation and blockchain-anchored roots.

---

9. The Consent Graph

A directed graph of consent relationships with indices for nodes, outgoing edges, incoming edges, edges by id, groups, and an audit log.

Edges are **scoped** (`EXECUTE`, `READ`, `LIST`, …), may carry **conditions** evaluated against request context, and may **expire**.

9.1 Delegation

`delegate()` enforces four invariants:

1. The parent edge must be valid (not expired, not revoked)
2. The parent must be marked `can_delegate()`
3. `effective_scopes.issubset(parent.scopes)` — **no scope expansion through delegation**
4. Depth must be within the configured limit

Invariant 3 is the structurally important one. A delegated grant can only ever be a subset of what was delegated, so authority strictly narrows as it propagates.

9.2 Cascading revocation

`revoke()` follows `parent_edge_id` links and revokes children transitively. Revoking a grant revokes everything that was derived from it — no orphaned delegated authority survives its source.

9.3 Four-step resolution

`evaluate()` tries, in order:

1. A direct edge from source to target
2. The target's group membership
3. The source's group membership
4. A BFS over delegation chains (`max_depth=10`), traversing only edges marked `delegatable`

It returns `ConsentDecision(granted, path, reason)`. The `path` matters: a grant obtained three delegation hops away is explainable rather than magical, and the same structure explains denials.

---

10. The Reflexive Core

Behavioural anomaly detection using per-actor, per-metric statistical baselines.

10.1 Sigma thresholds

```
_DEFAULT_SIGMA_THRESHOLDS = {
    LOW: 2.0, MEDIUM: 3.0, HIGH: 4.0, CRITICAL: 5.0,
}
```

Each `(actor, metric)` pair accumulates a running baseline. Deviations are classified by standard-deviation distance. `min_samples=10` prevents an actor's first few operations from generating noise before the baseline means anything.

One ordering detail carries real weight: **drift is checked before `baseline.update(value)`.** If the update came first, a large anomaly would partly absorb itself into the baseline it is being measured against, dulling detection precisely when it matters.

10.2 Escalation

`EscalationEngine.evaluate` gates each rule through three conditions in sequence:

1. `rule.matches(event)`
2. Cooldown has elapsed
3. `_trigger_counts[f"{actor_id}:{rule_id}"] >= rule.threshold_count`

The cooldown plus threshold combination is what makes escalation usable in production: a single transient spike does not suspend an agent, and a genuinely misbehaving actor still escalates promptly. `ESCALATION_TRIGGERED` is emitted at `CRITICAL`.

10.3 Introspection and pre-execution verdicts

The `IntrospectionEngine` converts an actor's behavioural state into an actionable verdict:

```
# HALT
if halt_actions & set(active_escalations):     return ExecutionVerdict.HALT   #
SUSPEND_AGENT / SHUTDOWN
if threat_level == ThreatLevel.CRITICAL:       return ExecutionVerdict.HALT
# THROTTLE
if threat_level == ThreatLevel.HIGH:           return ExecutionVerdict.THROTTLE
if THROTTLE in active_escalations:             return ExecutionVerdict.THROTTLE
# REQUIRE_CONFIRMATION
if threat_level == ThreatLevel.MEDIUM:         return ExecutionVerdict.REQUIRE_CONFIRMATION
if REQUIRE_CONFIRMATION in active_escalations: return ExecutionVerdict.REQUIRE_CONFIRMATION
return ExecutionVerdict.PROCEED
```

Two inputs — the numeric threat level and the set of currently-active escalations — collapse into one of four actions. The engine can also `bind_to_provenance`, so introspection results and the accountability log are themselves part of the tamper-evident record.

Detectors are pluggable, and an `ActorProfileManager` tracks per-actor scope usage and threat scores.

---

11. Certification and Attestation

A seven-step pipeline turns a declared manifest into a signed attestation:

```
validate → digest → determine level → build attestation
         → sign → publish to marketplace → emit event
```

Certification levels: `STRICT`, `STANDARD`, `BASIC`, `SELF_ATTESTED`, `UNCERTIFIED`, plus `UNSIGNED`.

11.1 The crypto requirement

`require_crypto_for_valid` defaults to **`True`**. With no crypto handler available, the result is `UNSIGNED`, `is_valid()` returns `False`, and — critically — **marketplace trust is not updated.** An unsigned attestation cannot influence trust scoring.

Setting it to `False` produces attestations that report as valid without a signature. The pipeline logs a warning, and the docstring states plainly that this is suitable only for development or test environments. It should never be `False` in production; an unsigned "valid" attestation is an unfalsifiable claim.

Default attestation validity is 90 days.

11.2 Level-to-trust mapping

```
STRICT        → auditor_verified
STANDARD      → community_verified
BASIC         → self_certified
SELF_ATTESTED → self_certified
UNCERTIFIED   → unverified
```

12. Trust Scoring

`TrustRegistry` maintains a composite score per tool:

```
DEFAULT_SCORE_WEIGHTS = {"certification": 0.50, "reputation": 0.35, "age": 0.15}
DEFAULT_AGE_HALF_SATURATION_DAYS = 30.0
```

Certification (0.50) — the dominant term, because a cryptographic attestation is stronger evidence than accumulated opinion.

Reputation (0.35) — starts at `0.5` (neutral, not trusted), moves by `event.impact * 0.1` per event, clamped to `[0, 1]`. New tools are neither trusted nor suspected.

Age (0.15) — `1.0 - math.exp(-age_days / tau)`. Exponential saturation, so age contributes quickly at first and then plateaus. Longevity is weak evidence of trustworthiness, and this shape refuses to let it become strong evidence.

Scores are recomputed on read rather than cached, so a certification or reputation change takes effect immediately. `TRUST_CHANGED` is emitted only when `abs(new - old) >= 0.05`, suppressing event noise from insignificant drift; a drop below `0.3` is emitted at `CRITICAL`.

The registry is in-memory (`dict[str, TrustRecord]` keyed by tool name).

13. Revocation and Federation

13.1 CRL

`CertificateRevocationList` supports revocation by tool name or attestation id, with typed reasons (`SECURITY_INCIDENT`, `MANUAL_REVOCATION`, …), emergency entries, propagated entries, and `unrevoke`. `is_revoked` / `is_attestation_revoked` are the hot-path queries.

13.2 Federation

`TrustFederation` lets independently-operated registries share evidence without merging authority. Peers have a `trust_weight` and a status (`ACTIVE`, `SYNCING`, …).

`query(tool_name)` collects the local score plus scores from peers that are active-or-syncing and meet `min_peer_weight`, then merges:

```
# local score gets weight 1.0; each peer contributes its configured trust_weight
return weighted_sum / total_weight   # 0.0 when total_weight == 0
```

The local registry always carries full weight; peers are advisory in proportion to how much they are trusted. **Revocation checks are strictly local** — `is_revoked` consults `self._local_crl`, so a peer cannot directly revoke on another operator's behalf. Peer revocations arrive through `receive_revocation` and enter the local CRL as propagated entries, keeping the local operator in the loop.

Revocation broadcast is available in both synchronous (`broadcast_revocation`) and asynchronous (`abroadcast_revocation`) form over a pluggable `BroadcastTransport` protocol.

---

14. Compliance Mapping

Six frameworks are modelled: **GDPR, HIPAA, SOC 2, ISO 27001, PCI-DSS, NIST 800-53**, alongside a default SecureMCP framework.

The model is `ComplianceFramework` → `ComplianceRequirement` → `RequirementCategory` (ten categories) → `ComplianceStatus`. Combined with the `Citation` type on policy results, this is what allows a compliance report to answer "which control does this denial satisfy, and under which version of the text?" as a query rather than as an exercise in institutional memory.

---

15. The Sandbox: What It Is and Is Not

Precision matters more here than anywhere else, because the word "sandbox" invites assumptions this implementation does not meet. The module's own docstring is unambiguous:

> There is no seccomp, no process isolation, no Python RestrictedPython, and no kernel-level boundary.

The sandbox is **cooperative**. Tools must call `runner.check(...)` to be governed. A tool that does not call it bypasses every declared policy.

What it does provide:

- Declarative permission manifests, mapped deny-by-default into an `ExecutionPolicy` via `policy_from_manifest`
- Drift visibility — divergence between declared and actual behaviour becomes observable
- Pre- and post-execution hooks
- A violation audit trail
- A CRL gate: `SandboxedRunner.start` refuses to start when `crl.is_revoked(manifest.tool_name)`

What it does not provide:

- Mandatory enforcement
- Protection from a deliberately malicious tool
- Syscall interception

The module recommends a real OS-level sandbox — seccomp plus namespaces on Linux, Seatbelt on macOS, AppContainer on Windows — with these checks layered on top as a second, semantic layer. That is the intended reading: SecureMCP's sandbox is a *declaration and observation* mechanism for tools that participate, not a containment boundary for tools that do not.

---

16. The Event Bus

`SecurityEventBus` exposes two emission paths with materially different guarantees.

`emit()` (sync) cannot interrupt a synchronous handler. When one exceeds `_DEFAULT_HANDLER_TIMEOUT_MS = 1_000`, the bus warns, increments `_slow_count`, and lets it finish — the honest behaviour, since pretending to time out a blocking call would not stop it. It refuses coroutine handlers outright, and defensively `close()`s any awaitable a sync-typed handler returns to avoid "never awaited" leaks.

`aemit()` (async) gathers handlers concurrently, hard-caps each with `asyncio.wait_for`, and runs sync handlers via `asyncio.to_thread` so they cannot block the loop.

This is why the policy engine uses `aemit`: on the request path, a slow subscriber must be *bounded*, not merely reported.

Counters for events, errors, slow handlers, and timeouts are exposed for operational monitoring — the bus reports on its own health rather than degrading silently.

---

17. Persistence

Storage is defined by a `Protocol`, not a base class:

```
@runtime_checkable
class StorageBackend(Protocol):
    ...
```

Three properties define the contract:

Synchronous. Deliberately. Storage calls happen inside middleware and event handlers; a sync interface keeps the backend implementable without an event loop, and `aemit`'s `to_thread` handling covers the async path.

JSON-safe dicts at the boundary. No ORM types cross the interface, so a backend can be a dict, a SQL table, or a remote service without changing callers.

Append-only versus mutable are distinct verbs. `append_*` for provenance records and exchange logs; `save_*` / `remove_*` for mutable state. The interface itself encodes which data is history and which is current state — a backend cannot accidentally offer `remove` semantics on the ledger.

Coverage: provenance, exchange log, contracts, behavioural baselines, drift events, escalations, the consent graph, marketplace state, policy versioning, workbench, and proposals.

Implementations: in-memory, SQLite, and PostgreSQL (with connection pooling). A single `backend` on `SecurityConfig` is propagated in `__post_init__` to the policy, contracts, provenance, reflexive, consent, and gateway layers, so one setting configures persistence for everything that needs it.

---

18. Cryptography

One function underpins every signature in the system:

```
def _canonicalize(data):
    return json.dumps(data, sort_keys=True, separators=(",", ":")).encode("utf-8")
```

Sorted keys, no whitespace. Without canonicalisation, two semantically identical payloads serialise differently and signatures become unverifiable across languages, versions, or dict ordering. Every signed artefact — contracts, attestations — is signed over this canonical form.

Supported algorithms:

Algorithm	Notes
HMAC-SHA256	Verified with `hmac.compare_digest` (constant-time)
RSA-PSS	MGF1-SHA256, `MAX_LENGTH` salt
ECDSA P-256	—

RSA and ECDSA require the optional `cryptography` package; HMAC works from the standard library alone. `verify_with_external_key` constructs a temporary handler from `AgentKeyRegistry` key material — this is the mechanism behind agent countersignatures in §7.1.

---

19. Observability Surfaces

Three ways to see what the governance layer is doing.

MCP tools (the gateway). `create_audit_tools` and `create_marketplace_tools` expose governance as *tools*, so an agent can query the ledger, drift events, the consent audit log, policy status, and overall security health through the same protocol it uses for everything else. Governance is introspectable by the agents being governed.

HTTP API. `mount_security_routes(server)` mounts a JSON API — dashboard, provenance (including chain status, Merkle proofs, export, and bundle verification), trust registry and scores, federation status, revocations, compliance reports, and an extensive policy surface: status, audit, simulate, schema, bundles, packs, environment profiles, promotions, versions, rollback, diff, snapshot import/export, analytics, and migration preview. Default `security_api_require_auth=True`, default prefix `/security`.

Dashboard. Always constructed; a read-only aggregate projection over whatever layers are active.

---

20. Threat Model and Honest Limits

Threats the architecture addresses:

- **Unauthorised invocation** — policy kernel, fail-closed, with list filtering so unauthorised capability is not even discoverable

- **Tampering with history** — hash chain plus Merkle tree plus randomised genesis, independently verifiable off-server
- **Authority creep** — consent delegation cannot expand scope; revocation cascades
- **Behavioural compromise** — sigma-based drift detection with graduated response
- **Supply-chain claims** — signed attestations; unsigned certification cannot move trust
- **Compromised-tool response** — CRL with federated propagation

The limits, stated plainly:

1. **The sandbox is not a boundary.** Cooperative only. Pair it with a real OS sandbox (§15).
2. **Contract term evaluation fails open.** `_evaluate_terms` accepts all terms if the evaluator raises, unlike the policy engine's fail-closed default. Policy runs first, which limits the blast radius, but the asymmetry is real (§7.2).
3. **`require_crypto_for_valid=False` is development-only.** It yields attestations that report valid without a signature (§11.1).
4. **`bypass_stdio=True` disables enforcement on STDIO.** Off by default, and warned about loudly when combined with policy.
5. **Unknown policy constraints are not enforced.** They are debug-logged and pass through. Treat those log lines as version skew (§5.2).
6. **Approval consumption is detected via substring matching** on a provider's free-text reason. Functional, but a structured field would be more durable (§6.1).
7. **Actor identity is a token prefix.** Eight characters is a correlation key, not an authenticated principal. Real principal identity comes from the auth layer.
8. **The trust registry is in-memory.** Trust scores do not survive a restart unless the backing marketplace state is persisted.
9. **Token counts are heuristic.** `chars // 4`, useful for volumetrics, not for billing.

One wiring defect is outstanding, and is recorded here rather than left to discovery. In `SecurityOrchestrator.bootstrap`, the consent block constructs `FederatedConsentGraph(..., federation=ctx.federation, ...)` before the federation block has run, so `ctx.federation` is still `None` at that point. Federated consent therefore does not receive a federation reference under the default bootstrap ordering. Remediation is to move federation construction ahead of the consent block, or to inject the reference once both exist.

---

21. Summary

SecureMCP's architecture reduces to a small number of decisions, applied consistently across the platform.

One bootstrap. A stateless factory turns config into an all-optional context. Fourteen layers, each independently enableable, one authoritative source of truth for what is on.

One request path. Five middleware in a fixed order — policy, contracts, provenance, reflexive, consent — each with a clear responsibility and each fail-closed on the enforcement path.

Two integrity structures. A hash chain for sequential tamper-evidence, a Merkle tree for inclusion proofs, and exportable bundles so a third party can verify both without trusting the server.

Explainability throughout. Cited policy decisions, path-carrying consent decisions, sigma-quantified drift events. Every outcome can be reconstructed after the fact.

Documented boundaries. The sandbox says it is not a kernel boundary. The dev-only crypto bypass says it is dev-only. The fail-open contract path is a known divergence rather than a hidden one.

That last property is what makes the rest usable. A governance layer is only as good as the accuracy of its claims — an operator who knows exactly where enforcement stops can put a real boundary there. Publishing that boundary precisely is a design obligation of this platform, not a caveat appended to it.